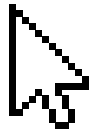
COE 454
Software Engineering II:

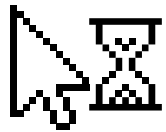
Open-Source Licensing



Presented by Eliel Keelson

Why Open-Source Licences Exist

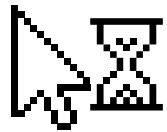
- Copyright applies automatically the moment you write code. By default, all rights are reserved, nobody can copy, modify, or distribute your work without your explicit permission.
- That default position makes collaboration impossible at scale. If every developer had to negotiate individual permission with every user, the modern internet could not exist.



Why Open-Source Licences Exist

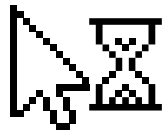
- Open-Source Licenses solve this by granting permission in advance, to everyone, under defined conditions. The author attaches a license once; users accept by using the software.

- The core philosophical question that divided the movement:
- The copyleft tradition (Richard Stallman, 1983): ‘free’ means freedom, not price. Any software built on free software must itself be free. Freedom propagates forward.

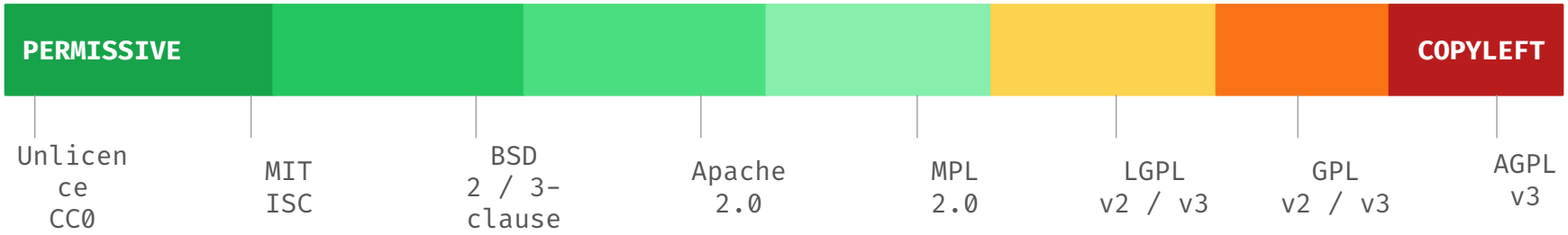


Why Open-Source Licences Exist

- **The permissive tradition:** attach as few conditions as possible. The software spreads further, more people benefit, and everyone is better off. Freedom means no strings attached.
- *Almost every license in existence sits somewhere between these two poles. Understanding where each sits is the practical skill.*



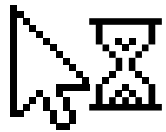
The Licensing Spectrum



- **Permissive licenses (left side):** use the code for anything, including commercial products, with minimal obligations. Ideal for broad adoption.
- **Weak copyleft (middle):** some sharing obligations at the file or library level, but proprietary products can still include these components.

The Licensing Spectrum

- **Strong copyleft (right side):** derivative works and, in AGPL's case, network services must also be open-sourced. Incompatible with closed commercial products.
- *Your choice of license for dependencies is an architectural decision. Make it deliberately and record it in your ADR.*



Brief History of Open-Source Licensing

1983 Richard Stallman launches the GNU Project to build a free Unix-like operating system, beginning the copyleft tradition

1989 GPL Version 1 published, the first formal copyleft licence

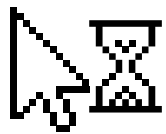
1991 GPL v2 and LGPL published. Linux kernel released under GPL v2, making copyleft the licence of the world's most important operating system

1998 The term 'open source' coined in Palo Alto. Open Source Initiative (OSI) founded to certify compliant licences

2004 Apache Licence 2.0 published, adds patent grant to permissive licensing. Becomes dominant in enterprise open source

2007 GPL v3 and AGPL v3 published, closing the patent loophole and the application-service-provider loophole respectively

2018–2023 MongoDB (SSPL), Elasticsearch, HashiCorp (BUSL) abandon traditional open-source licences to prevent cloud-provider exploitation, an unresolved debate continues



Permissive Licences

MIT License

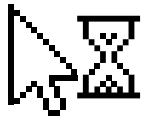
Massachusetts Institute of Technology License

Do whatever you want with this code. Just keep the original copyright notice.

- **Origin:** Developed at MIT for the release of the X Window System, the graphical windowing environment underpinning most Unix desktop interfaces. Now the most widely used open-source license in the world.

- **What it permits:**

Commercial use, modification, distribution, private use, sublicensing, all freely permitted. You may sell a product built on MIT-licensed code without restriction.



Apache 2.0

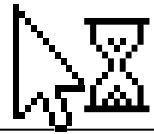
Apache License, Version 2.0

Like MIT, but with an explicit patent grant and a notice-of-modifications requirement.

- **Origin:** Published by the Apache Software Foundation in January 2004. The ASF was founded in 1999 to support the Apache HTTP Server, which its founders described as ‘a patchy server’, a play on words that became the name.

- **What it adds beyond MIT:** Patent grant: contributors explicitly license any patents they hold that are implemented in the software, protecting users from later patent claims by the same contributor.

- **Notice of modifications:** if you modify Apache-licensed files and distribute the result, you must state that you have modified them. You do not need to release the modifications.



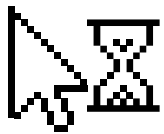
Apache 2.0

- **Compatibility note:**

Apache 2.0 is compatible with GPL v3 but NOT with GPL v2. You can combine Apache 2.0 and GPL v3 code in a project released under GPL v3; you cannot do the same with GPL v2 code.

- **Used by:**

Android, Apache HTTP Server, Kubernetes, Swift (Apple), Hadoop, Kafka, and the majority of Google open-source projects.



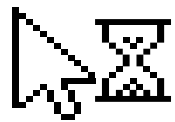
BSD Licenses

Berkeley Software Distribution Licenses (2-clause and 3-clause)

The grandfathers of permissive licensing, functionally close to MIT, with two variants.

- **Origin:** BSD Unix was developed at the University of California, Berkeley, from the 1970s. Its license was one of the first widely used open-source licenses. An original 4-clause version included an advertising requirement that became impractical as adoption spread.

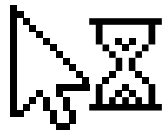
- **3-clause BSD (Modified BSD, New BSD):** Requires: (1) keep the copyright notice in source redistributions, (2) keep it in binary redistributions, (3) do not use the project's name to endorse derived products without permission. Used by FreeBSD, OpenBSD, NetBSD.



BSD Licenses

- **2-clause BSD (Simplified BSD, FreeBSD License):**
Removes the endorsement clause. Functionally nearly identical to the MIT license. Many legal analysts consider them equivalent for practical purposes.

- **Used by:**
FreeBSD (the OS underlying MacOS and PlayStation systems), Dart (Flutter’s language), Go’s standard library, LLVM (compiler infrastructure underlying Apple’s development tools).



ISC (Internet Systems Consortium) License

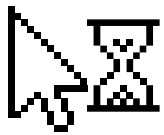
Internet Systems Consortium License

The shortest major permissive license. Functionally identical to MIT

- **Origin:** Developed by the Internet Systems Consortium, an American non-profit that manages critical internet infrastructure including the BIND DNS server, the software that routes much of the internet's traffic.

- **What it is:**

At approximately 130 words, the ISC license is even shorter than MIT. It permits any use, modification, and distribution with the single requirement of preserving the copyright notice. There are no meaningful legal differences from MIT for practical purposes.



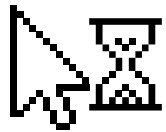
ISC (Internet Systems Consortium) License

- **Why it matters to you:**

A significant number of npm packages use the ISC license, particularly those written by developers who prefer brevity. When you run npx license-checker on your project, ISC will appear frequently alongside MIT.

- **Used by:**

OpenBSD, BIND (the DNS server), and many npm packages. ISC and MIT can be treated identically from a commercial licensing standpoint.




Weak Copyleft Licences

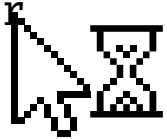
MPL 2.0

Mozilla Public License, Version 2.0

Copyleft at the file level; not the project level. The pragmatic middle ground.

- **Origin:** Created by the Mozilla Foundation, the non-profit behind the Firefox browser, as a deliberate middle ground between permissive licenses and the strong copyleft of the GPL. MPL 1.0 appeared in 2000; version 2.0 simplified and improved it in 2012.

- **The key innovation, file-level copyleft:** If you take an MPL-licensed file, modify it, and distribute the result, that modified file must be released under the MPL. But you can combine MPL-licensed files with files under other licenses, including proprietary licenses, in the same project, and the other files are not affected.

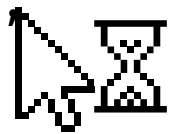


MPL 2.0

- **Practical implication:** A company can take an MPL-licensed library, modify some of its files, and include it in a proprietary product. The modified MPL files must be open-sourced; the rest of the proprietary product does not. This makes MPL compatible with commercial development in a way that GPL is not.

- **Used by:**

Mozilla Firefox, Thunderbird, LibreOffice (partially) and the Rust programming language's standard library.

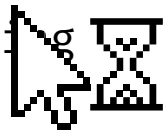


LGPL (GNU Lesser Public License, v2 and v3)

Link freely from proprietary code. Modify the library itself and you must share those changes.

- **Origin:** Originally called the Library General Public License and published in 1991 alongside GPL v2. The name was later changed to ‘Lesser’ to reflect Stallman’s view that it represents a lesser degree of freedom than the full GPL. The current version is LGPL v3, published in 2007.

- **Why it was created:** The strict GPL created an adoption problem: a free C library that could only be used in other GPL projects would never reach the mainstream. LGPL solved this by allowing proprietary applications to link to LGPL libraries without triggering copyleft, while still requiring modifications to the library itself to be shared.



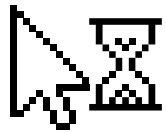
LGPL (GNU Lesser Public Licence, v2 and v3)

- **The critical distinction:**

Linking to an LGPL library from proprietary software: permitted without copyleft obligation.
Modifying the LGPL library itself and distributing the result: those modifications must be released under LGPL.

- **Used by:**

The GNU C Library (glibc), the reason commercial applications can run on Linux without opening their source code. Qt framework (under a dual licence). GTK (the GNOME graphical toolkit).




Strong Copyleft Licences

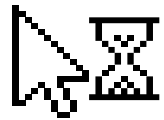
GPL (GNU General Public License, v2 and v3)

Freedom is viral. Any software built on GPL code and distributed must itself be GPL.

- **Origin:** Written by Richard Stallman in 1989. GPL v2 (1991) became the license of the Linux kernel, making it the license governing the most widely deployed operating system in history. GPL v3 (2007) added provisions addressing software patents and digital rights management that v2 had left unaddressed.

- **The copyleft mechanism:**

GPL uses copyright law against itself. Instead of restricting what people can do, it restricts what restrictions they can impose. If you distribute a modified GPL program, the entire distributed work must also be under the GPL, including making the source code available to recipients.

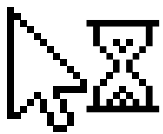


GPL (GNU General Public License, v2 and v3)

- **The commercial implication:** If your product includes a single GPL-licensed library and you distribute it to clients, your entire product's source code becomes subject to the GPL. Clients can legally demand it. Competitors can take it, modify it, and release their own version. For most commercial software firms, this is unacceptable.

- **Used by:**

Linux kernel (GPL v2), WordPress (GPL v2), Git (GPL v2), GCC, the GNU Compiler Collection (GPL v3), Bash (GPL v3), MySQL (GPL v2 with commercial alternative).

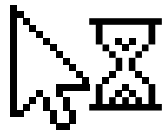


AGPL (GNU Affero General Public License, v3)

The GPL loophole for SaaS is closed. Run it as a service, and you must share your source.

- **Origin and the loophole it closed:**

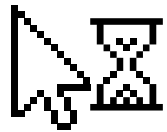
The GPL required sharing source code when you distribute software. But web applications are not distributed, you run them on a server and users interact through a browser. Companies could take GPL software, modify it, run it as a SaaS product, and never trigger the GPL's sharing requirement. Affero Inc. created an early version of this license in 2002 to close this gap; the GNU project standardized it as AGPL v3 in 2007.



AGPL (GNU Affero General Public License, v3)

- **What it adds to GPL:** If you run AGPL-licensed software as a network service and users interact with it remotely, you must offer those users access to the corresponding source code, including any modifications you have made.

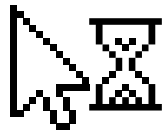
- **The practical implication for SaaS developers:** If any component of your cloud-hosted application is AGPL-licensed, you may be required to publish your entire application's source code. This is the license designed specifically to prevent AWS, Google, and Microsoft from running private, improved forks of open-source projects.



AGPL (GNU Affero General Public License, v3)

- **Used by:**

Mastodon (decentralized social network), Nextcloud, many self-hosted tools that need to prevent cloud exploitation.



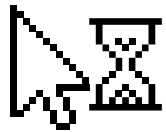

Creative Commons Licences

Creative Commons (CC) – What the Abbreviations Mean

CC stands for Creative Commons – a non-profit founded in 2001 by Lawrence Lessig to build a body of creative works the public can freely use and build upon.

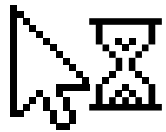
Critical rule: Creative Commons licenses are NOT recommended for software. The CC organization says so explicitly. They lack patent provisions and use language designed for creative works, not code.

Where you will encounter them as a software engineer: documentation, written content, educational resources, datasets, icons, fonts, images, and design assets.



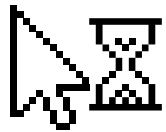
Creative Commons Zero (CC0)

The author waives all copyright and places the work in the public domain. No attribution required. No restrictions. Freely usable for any purpose including commercial. Equivalent to the license for software.



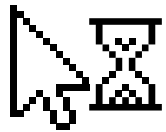
Attribution-ShareAlike (CC BY-SA)

Credit the author, and release any derivative works under the same CC BY-SA license. The copyleft version of CC. SA = ShareAlike.



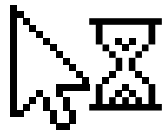
Attribution-NonCommercial (CC BY-NC)

May not be used for commercial purposes. NC = NonCommercial. A CC BY-NC image CANNOT go into a commercial product without the author’s separate written permission.



Attribution-NoDerivatives (CC BY-ND)

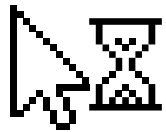
You may use and redistribute unchanged, but you may not modify it. ND = NoDerivatives.



Dual Licensing – The Commercial Model

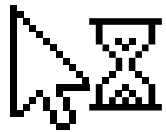
Dual Licensing is the practice of releasing the same software under two licenses simultaneously, a copyleft license for open-source use and a commercial license for proprietary use.

Users choose based on their situation: open-source projects take the GPL version at no cost; commercial developers who cannot accept GPL conditions purchase a commercial license.



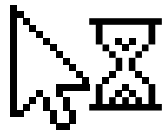
Dual Licensing – The Commercial Model

- **MySQL – Most Famous Example:**
Available under GPL v2 for use in other GPL projects. Companies building proprietary products pay Oracle (who acquired MySQL through Sun Microsystems) for a commercial license. This model allows the project to generate revenue from commercial users while remaining free for the open-source community.



Dual Licensing – The Commercial Model

- **Qt – The Cross-Platform UI Framework:**
Available under LGPL for open-source projects and under a commercial license for proprietary applications. Widely used in embedded systems, automotive interfaces, and desktop applications.

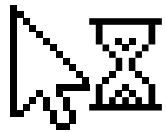


Dual Licensing – The Commercial Model

- **The business logic:**

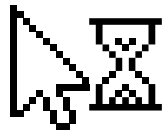
If a project is useful enough that commercial developers must use it, they will pay for the privilege of doing so without the copyleft obligation. The copyleft version acts as a marketing tool for the commercial license.

For your own projects post-graduation: dual licensing is a viable business model for libraries and frameworks with commercial appeal.



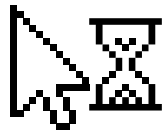
The New Debate — Source-Available Licenses

From 2018 onwards, several companies whose open-source projects became foundations of major cloud provider revenues abandoned traditional open-source licenses entirely.



The New Debate – Source-Available Licenses

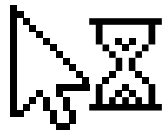
- **SSPL, Server Side Public License (MongoDB, 2018):** Inspired by AGPL but goes significantly further. If you offer SSPL-licensed software as a service, you must open-source not just your modifications but your entire software stack, the orchestration, monitoring, storage, and networking layers. Specifically designed to prevent AWS and others from offering managed MongoDB without contributing back. The Open Source Initiative has NOT approved SSPL as an open-source license.



The New Debate – Source-Available Licenses

- **BUSL, Business Source License (MariaDB, 2016; HashiCorp Terraform, 2023):**

The code is source-available but cannot be used in production for commercial purposes for a defined period, typically four years. After that it converts to a proper open-source licence (usually GPL or Apache 2.0). This allows a company to benefit from community scrutiny of its code while preventing immediate commercial exploitation.

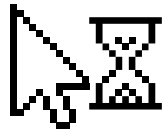


The New Debate – Source-Available Licenses

- **The unresolved question:**

These licenses are not open source by the OSI definition. But the companies that adopted them argue that pure open source is not financially sustainable when hyper scale cloud providers can take, improve, and sell your software without contributing back.

As a software engineer, you will encounter these licenses. Know that they carry commercial restrictions that MIT, Apache 2.0, and GPL do not.



Licensing in Practice – Common Mistake vs Best Practice

✗ Common Mistake:

- A team builds an inventory system for a Kumasi shop. They use a popular charting library that turns out to be GPL v3 licensed.
- They do not check. They deliver the system to the client as closed-source commercial software.
- Technically, the entire delivered system is now subject to GPL v3. The client could legally demand the source code. A competitor could request it and build a competing product.
- Nobody discovers this, until they try to sell the product to a second client, at which point a lawyer flags the problem.
- The fix requires replacing the GPL library with an MIT alternative and rewriting the affected module. Two weeks of unplanned rework.

Licensing in Practice – Common Mistake vs Best Practice

✓ **Best Practice:**

- Before writing any code, the team discusses which libraries they intend to use and checks their licenses in the ADR.
- They choose a MIT-licensed charting library as an alternative, noting the decision and rationale in ADR-003.
- Before final delivery, they run npx license-checker --summary and confirm no GPL or AGPL dependencies remain.
- The license audit report is attached to the final handover package. The client signs the MVP Acceptance Form with full knowledge of the licensing status.
- When they later wish to sell the product commercially, the licensing is clean. No unplanned rework. No legal exposure.

Practical Licensing Checklist – Before You Ship

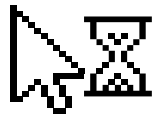
1. For your dependencies (what you are building on):

- Run npx license-checker --summary (Node.js) or pip-licenses (Python) to list every dependency license
- Flag any GPL or AGPL dependency immediately, determine whether it is compatible with your commercial intent
- Document your license choices in your Architecture Decision Record as a permanent record
- If in doubt about compatibility, find an MIT or Apache 2.0 alternative before building further

Practical Licensing Checklist – Before You Ship

2. For your own code (what you are releasing or delivering):

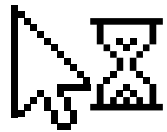
- Commercial product for a client: use MIT or Apache 2.0 for any components you will reuse; the IP assignment clause handles the rest
- Internal tool, never distributed: license does not practically matter, but document it anyway
- Library you want widely adopted: MIT or Apache 2.0 for maximum reach
- Tool where you want contributions to remain open: GPL v3 (application) or LGPL v3 (library)
- SaaS product you want to protect from cloud exploitation: AGPL v3, with legal advice



Practical Licensing Checklist – Before You Ship

3. For assets (icons, fonts, images, documentation):

- CC0 or MIT-licensed assets are safe for commercial products
- CC BY-NC assets cannot be used commercially without separate permission, find an alternative

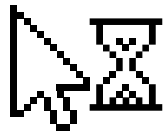


About Open-Source Licensing

- Have you run a license audit on your current semester project? If not, do it before the end of this week. What licenses do your dependencies carry?

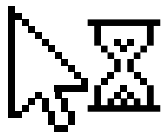
- Your team's project uses a GPL-licensed library. Your client has asked whether they can sell the product commercially after you graduate. What do you tell them?

- If you wanted to release a library you have built so that other developers can use it freely but companies must pay for commercial use, which licensing model would you choose and why?



Things to consider

- A popular npm package you depend on switches its license from MIT to AGPL v3 in its next version. You are mid-development. What are your options, and which would you choose?
- When you hand over your semester project to your client, will you include a document listing the open-source components it contains and their licenses? If not, should you?



Open-Source Licensing – The One Slide Summary

License	Type	In plain language
MIT / ISC	Permissive	Use for anything. Keep copyright notice. Safe for all commercial products.
Apache 2.0	Permissive+	Like MIT, plus patent grant. Preferred for enterprise and patent-sensitive domains.
BSD 2/3-clause	Permissive	Effectively equivalent to MIT. Used widely in BSD Unix derivatives and Flutter.
MPL 2.0	Weak Copyleft	File-level copyleft. Modified files open-sourced; rest of product can be proprietary.
LGPL v2/v3	Weak Copyleft	Link to the library freely. Modify the library itself and share those changes.
GPL v2/v3	Strong Copyleft	Distributed products containing GPL code must be entirely GPL. Incompatible with closed products.
AGPL v3	Strongest	Like GPL, but network use triggers copyleft. SaaS products must open-source their stack.
CC licences	Assets only	Not for software. For documentation, images, datasets. Check NC (NonCommercial) carefully.
SSPL / BUSL	Source-available	Not OSI-approved open source. Commercial restrictions apply. Read carefully before using.